

Kaaval Assurance

**Routers predict.
Kaaval verifies.**

An assurance layer for AI decisions — checked locally on AMD compute before anyone downstream sees the answer.

VERIFY, NOT PREDICT. RECEIPTS, NOT PROMISES.

ONE PUBLIC CONTAINER

**AMD
+ Gemma
+ ROCm / vLLM**

Evidence Baseline — no credentials
Live Session — BYOK when needed

RECEIPTS, NOT PROMISES

A tribunal already rejected “the AI is separate.”

Moffatt v. Air Canada, BC Civil Resolution Tribunal, Feb 2024 — a real ruling, not a projection.

WHAT HAPPENED

Air Canada's chatbot told a customer he could book at full fare and claim a bereavement discount *after* travel. The real policy required the request *before* travel — the chatbot's answer was fluent, confident, and wrong.

Air Canada argued the chatbot was “a separate legal entity responsible for its own actions.” The tribunal rejected that outright and held the airline liable: \$812 CAD.

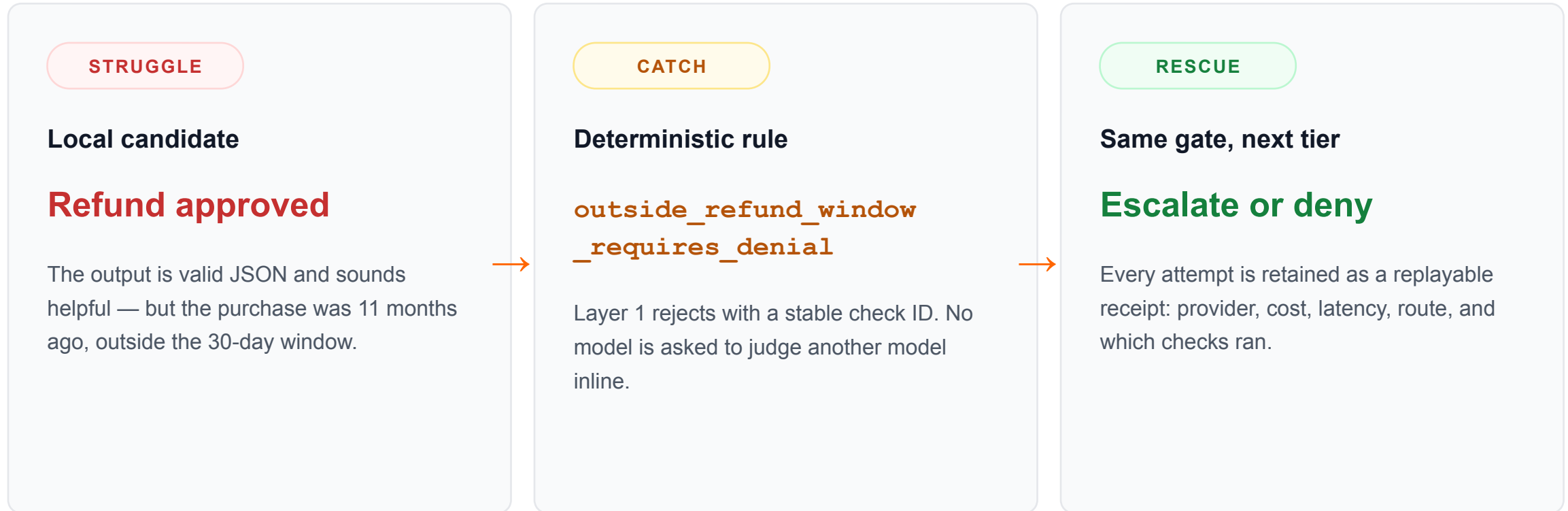
THE PATTERN KAAVAL GATES

The correct answer wasn't unknowable — it was sitting in a linked policy page in the same conversation. A fluent model still produced the wrong one.

This is exactly the shape of failure a deterministic contract check catches: not “is this text safe,” but “**does this claim match the policy on file.**”

Same shape of failure, caught before it ships.

A real contract from this repo: `support.refund_decision` — the \$500 cap is a range check, not a prompt.



Not a content filter. A guardrail blocks bad text. Kaaval decides which model answers — and proves what it cost.

One gate turns model traffic into governed decisions.

The request path stays deterministic; deeper audit remains sampled and offline.

1

Conformance Gate

Deterministic JSON shape, enums, ranges, and grounding checks. Pure code — no model judges another model inline.

2

Adaptive EWMA Routing

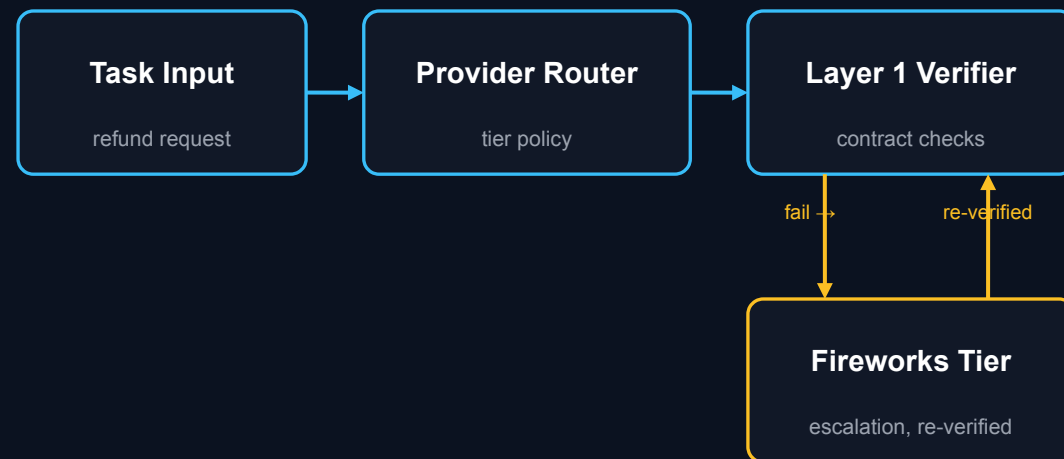
Tracks per-category contract-failure drift in real time; tightens or pre-routes to remote when a category keeps failing.

3

Sampled Offline Audit

FP-calibrated challenger samples accepted answers. Display-only — two-sided calibration and audit-to-routing are roadmap work, not shipped.

INFERENCE ASSURANCE FLOW



Only Layer 1 gates acceptance. Provider errors and double failure return NO SAFE ANSWER.

EVIDENCE, SEPARATED HONESTLY

AMD proves execution. Fireworks proves selective economics.

Two artifacts, two claims — never blended into one result.

16 / 16

MEASURED

Local Contract-Conformance

Layer 1 · 16-case eval

324.6 / 479.6ms

MEASURED

Latency P50 / P95

16-case eval

14 / 16

MEASURED

Remote Calls Avoided

87.5% reduction

\$0.0333

CONFIGURED

Configured Cost Avoided

88.7% reduction

AMD/ATI host · 47.98 GiB VRAM · gfx1100 target · gemma-3-1b-it via ROCm + vLLM

Exact GPU marketing name unavailable; none inferred.

Configured-cost figure is a price estimate from recorded token counts, not a provider invoice — tagged CONFIGURED, not MEASURED, on purpose.

`bundle live-5be3acfa-amd-gemma-proof · checksummed`

Pull one public container. Evidence appears immediately.

Connect a model endpoint only when you want live assurance.

`ghcr.io/kaaval-ai/kaaval-assurance:act-ii`

linux/amd64 · publicly pullable · clean-smoke verified

- ✓ Evidence Baseline — measured AMD bundle, zero credentials
- ✓ Live Session — Fireworks BYOK or reachable Ollama / vLLM
- ✓ Safe defaults — no model download; paid remote and export closed
- ✓ Acceptance — 361 tests · linux/amd64 · UID 10001 (non-root)

NOW

Provider-neutral gateway

Observe decisions · Evaluate contracts · Produce receipts

30 DAYS

Validation before expansion

Blind semantic benchmark · Two-sided Layer 3 calibration · TCO experiment

DESIGN PARTNERS

Shadow-mode proof

Refunds and support · False accept / reject rates · Operational economics

Pull one container. Inspect measured AMD evidence. Connect your endpoint when you want live assurance.

Kaaval Assurance · Verify, don't predict. Receipts, not promises.